DeepWiki Ip-59/TeachAI

Edit Wiki Share

Ask Devin about Ip-59/TeachAI

Fast

Menu

Getting Started & Configuration

Relevant source files

Purpose and Scope

This document details the initialization, installation requirements, configuration lifecycle, connection modes, and execution entrypoints for TeachAI. It bridges natural language configuration concepts (such as API credentials and proxying) with the underlying Python implementation (`config.py`).

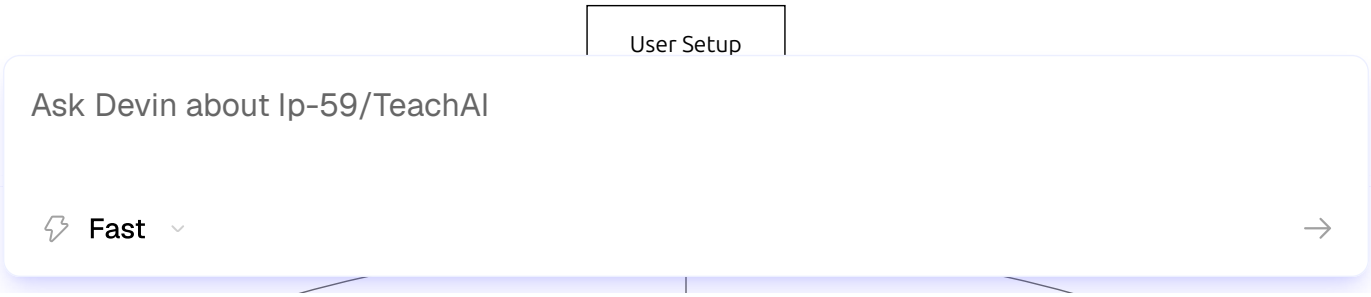
---

1. Environment Requirements & Dependencies

TeachAI relies on specific core packages for Jupyter notebook integration, asynchronous networking via **`httpx`** , and OpenAI API communication. Specialized educational modules (Data Analysis, Machine Learning, Web Development, and Finance) pull in respective scientific libraries.

The installation dependencies are cataloged in `requirements.txt` | `NaN-NaN` :

- **Core Engine & API**: **`openai>=1.95.1`** , **`httpx>=0.27.0`** , **`python-dotenv>=1.1.1`**
  `requirements.txt` | `2-8`
- **Jupyter & UI**: **`ipywidgets>=8.1.7`** , **`jupyter>=1.1.0`** , **`ipython>=9.4.0`** , **`ipykernel>=6.30.0`**
  `requirements.txt` | `4-7`
- **Rendering**: **`markdown>=3.5.2`** , **`pygments>=2.17.2`** , **`latex2mathml>=3.77.0`**
  `requirements.txt` | `11-13`
- **Domain Sub-stacks**: **`pandas`** , **`scikit-learn`** , **`flask`** , **`yfinance`** , etc.
  `requirements.txt` | `16-32`



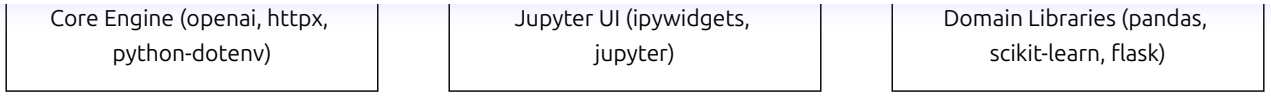


Figure 1.1.1: Dependency installation workflow.

Sources: `requirements.txt` | `1-32`

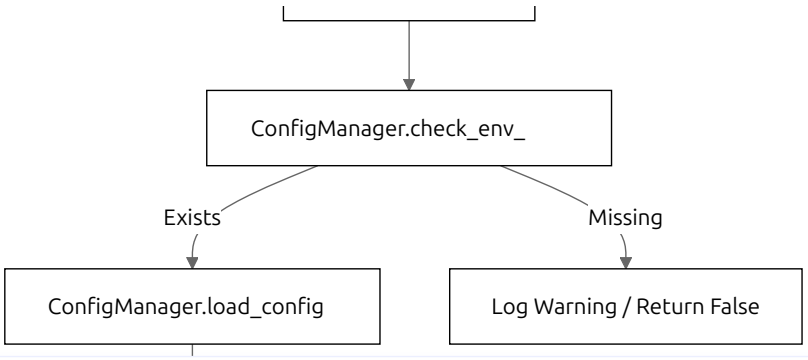
---

2. Configuration Management & `.env` Workflow

TeachAI uses environment variables managed by the `ConfigManager` class (**`[config.py:111-174]`**). Users must provision a local **`.env`** file based on the provided template `.env.sample` (`.env.sample` | `1-19`).

The Configuration Lifecycle

1. **Sample Generation**: **`ConfigManager.create_sample_env()`** writes default placeholders [config.py:195-208] to `.env.sample`.
2. **Loading**: **`ConfigManager.load_config()`** loads variables via **`python-dotenv`** (**`[config.py:133-147]`**).
3. **Validation**: Checks for mandatory keys such as **`OPENAI_API_KEY`** (**`[config.py:151-158]`**).
4. **Environment Application**: Invokes **`apply_openai_connection_environment()`** to reconcile proxy and connection mode settings (`config.py` | `148`).



Ask Devin about Ip-59/TeachAI
Fast

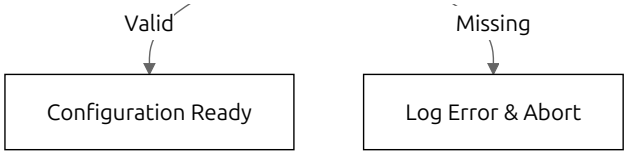


Figure 1.1.2: Configuration loading and validation flow bridging system actions to `ConfigManager`.

Sources: `config.py` | `111-208` , `.env.sample` | `1-19`

---

3. OpenAI Connection Modes: Direct vs. Proxy

TeachAI supports two distinct network integration modes for communicating with the OpenAI API: **`direct`** and **`proxy`** (`config.py` | `36-64`).

Mode Resolution Logic

- **`direct` (Default)**: Applications connect directly without internal HTTP proxies. System-level VPNs or direct networks handle routing. **`apply_openai_connection_environment()`** actively strips proxy keys (**`HTTP_PROXY`** , **`HTTPS_PROXY`** , **`OPENAI_PROXY`** , etc.) from **`os.environ`** to prevent interference from shell configurations or IDE environments (`config.py` | `79-83`).
- **`proxy`** : Explicitly enabled via **`OPENAI_CONNECTION_MODE=proxy`** or **`OPENAI_USE_PROXY=true`** (`config.py` | `48-52`). Requires a valid **`OPENAI_PROXY`** URL string (`config.py` | `161-166`).

Environment Variable	Allowed Values	Code Handler Function		Behavior
`OPENAI_CONNECTION_MODE`	**`direct`** , **`proxy`** , **`vpn`** , **`auto`**	`config.py`	`NaN-NaN`	Sets primary connection routing archetype [config.py:36-64]

Ask Devin about Ip-59/TeachAI

Fast

`user:pass@host:port`)	fig.py	NaN-NaN	client [config.py:27-33]

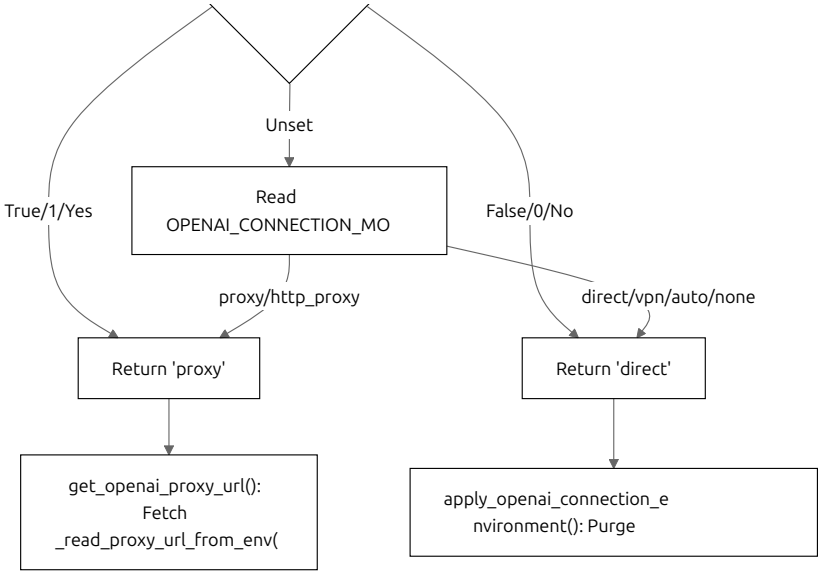


Figure 1.1.3: OpenAI connection mode resolution architecture.

Sources: config.py | 11-109

---

4. Course Catalog & Data Isolation (`courses.json`)

The course structure and metadata catalog are maintained in **`courses.json`** . To prevent student progress or local execution data from leaking into version control, repository isolation rules are defined in `.gitignore` .

Ask Devin about Ip-59/TeachAI

Fast

keys are strictly excluded (`[.gitignore:6-9]`).

Figure 1.1.4: Repository data exclusion and exception hierarchy.

Sources: `.gitignore` | `1-43`

---

5. Execution Entrypoints (Notebook & CLI)

TeachAI is designed to run interactively inside Jupyter environments (**`TeachAI.ipynb`**) or via programmatic entry scripts (**`run_teachai.py`**).

Startup Initialization Steps

1. **Environment Load**: The runner initializes **`ConfigManager`** to load credentials from **`.env`** (`config.py` | `133-147`).
2. **Engine Bootstrap**: Instantiates **`TeachAIEngine`** , validating course files and state stores (`lesson_interface.py` | `25-48`).
3. **Interface Mounting**: Renders widgets via **`ipywidgets`** inside the Jupyter kernel or launches the dashboard interface.

Sources: `config.py` | `111-174` , `lesson_interface.py` | `22-48` , `.env.sample` | `1-19`

Ask Devin about Ip-59/TeachAI

Fast

Ask Devin about Ip-59/TeachAI

Fast

Ask Devin about Ip-59/TeachAI

Fast

Ask Devin about Ip-59/TeachAI

Fast